

智能体 AI 的攻击与防御版图：一篇综合性综述

原文：The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey

作者：Juhee Kim、Xiaoyuan Liu、Zhun Wang、Shi Qiu、Bo Li、Wenbo Guo、Dawn Song

说明：这是一份以个人阅读体验为优先的中文译稿。正文按原论文结构组织，但不复刻双栏版式；文中的 [数字] 沿用原论文参考文献编号。图表不重绘，只在相应位置给出中文说明。参考文献书目信息不逐条翻译，可直接查阅原 PDF。

摘要

把大语言模型与非 AI 系统组件结合起来的 AI 智能体，正迅速进入真实应用，为自动化和灵活性带来了前所未有的提升。但这种灵活性也引入了复杂的安全问题，而且这些问题与传统软件系统中的安全问题有根本区别。

本文首次对 AI 智能体安全进行系统而全面的综述，内容包括：安全智能体系统的设计空间、攻击版图和防御机制。作者还通过案例研究指出当前智能体 AI 系统的安全缺口，并总结这一新兴领域尚待解决的问题。论文提出了首个用于系统理解智能体安全风险与防御策略的框架，希望为构建安全的智能体系统和推进相关研究打下基础。

关键词：AI 智能体、大语言模型、安全、提示注入、知识体系化。

1. 引言

智能体 AI 的快速发展正在改变整个 AI 领域：AI 已经从孤立的语言模型，转向把大语言模型与多种软件组件结合在一起的混合系统。这类系统把 AI 推理与传统软件无缝衔接，能够动态调用工具并自主完成任务，因此被用于聊天机器人 [48,111]、软件开发 [26,30,47]、网页浏览自动化 [122] 等大量场景。

与此同时，智能体系统的安全问题也越来越紧迫。近期事件已经展示了这些漏洞的严重程度：提示注入被用来访问私有 GitHub 仓库 [60]；远程代码执行漏洞让攻击者获得未

经授权的系统访问权限 [95,97]；恶意文档附件和邮件转发被用于窃取敏感数据 [125,128]；某些服务器暴露了用户聊天记录和凭据 [110,132]；攻击者甚至开始利用网页智能体进入用户的个人银行账户 [17]。换言之，智能体的灵活性和自动化能力越强，安全问题就越复杂，而且不能简单套用传统软件或独立 AI 模型的安全思路。

已有研究虽然对智能体安全作出了重要贡献，但大多只关注某一种攻击向量或单个系统组件，尤其集中于提示注入 [15,89,179,183,185]。这些工作缺少把智能体作为完整系统来防护的视角。我们需要一个系统性框架，用来理解多个组件组合之后会产生哪些新攻击面，以及为什么它们需要不同于组件级防御的安全方法。

本文从整体系统视角填补这一空白。作者首先系统刻画影响安全属性的智能体设计维度；随后整理整个智能体生态中的攻击向量与安全风险；再按保护方式归纳现有防御机制，并指出当前策略的关键缺口；最后分析多个真实智能体，并以 AutoGPT 为详细案例，进一步展示实际防御中的不足。

论文的主要贡献有三点：

1. **智能体设计维度。** 提出一个由七个关键维度组成的系统框架：输入信任度、访问敏感度、工作流、动作、记忆、工具和用户界面；分析每个维度的灵活性如何改变安全风险，并把这些维度映射到 MITRE ATLAS、OWASP LLM Top 10 等既有框架。
2. **完整的攻击版图与分类体系。** 按威胁模型把攻击者分为外部攻击者、用户级攻击者和内部攻击者；同时提出横跨 CIA 三要素（机密性、完整性、可用性）的七类安全风险，并分析风险之间如何在系统层面相互作用、彼此放大。
3. **防御版图的体系化。** 系统梳理已有防御机制，开展多项案例研究，归纳防御设计维度，并指出开放问题。

据作者所知，这是第一项从完整系统角度系统分析智能体 AI 安全版图的工作。它既可作为安全研究的基础框架，也可作为智能体研究人员和开发者的参考手册。

2. 综述范围与方法

2.1 范围

本文关注两类问题：只在智能体系统中出现的安全风险，以及相较传统软件或独立 LLM，在智能体中被显著放大的风险。

论文先从七个设计维度说明智能体与独立模型的区别（第 3 节），再从组件和系统两个层面分析智能体特有的风险（第 4 节）。对越狱、幻觉等同样存在于非智能体 LLM 中的问题，重点讨论智能体的自主性和环境访问能力如何把影响放大成数据外泄、非预期系统

修改等真实后果。模型反演 [40] 等以模型内部为目标、且在纯推理接口下不会因为智能化而根本恶化的攻击，不在本文范围内。第 5 节则归纳防御方法和开放问题。

2.2 方法

作者先根据现有定义和真实智能体实现，确定智能体结构与设计维度，再以此建立同时覆盖组件级和系统级风险的分类体系。分类过程也参考了 OWASP LLM 应用 Top 10 [118] 和 MITRE ATLAS 矩阵 [98]。

防御部分借鉴传统系统安全中的“纵深防御”原则。作者系统检索了 2023 年至 2025 年 10 月间的学术论文和网络资料，这一时期恰好对应智能体系统的大规模兴起 [178]。检索词分为四组：

- 通用术语，如 agent security；
- OWASP 风险，如 prompt injection、memory poisoning；
- 组件相关术语，如 RAG security、tool security；
- 传统安全方法的迁移，如 isolation、access control。

资料优先覆盖 USENIX Security、IEEE S&P、CCS、NDSS 等顶级安全会议，以及 NeurIPS、ICLR、ICML 等机器学习会议，同时纳入高影响力预印本、产业白皮书和 CVE。作者人工排除只研究独立模型的工作，最终得到 128 篇文献，其中 51 篇提出攻击方法，60 篇提出防御方法，其余同时涉及攻防，或只做案例研究而未提出新方法。

2.3 与既有综述的区别

已有 AI/LLM 安全综述主要强调模型级威胁，常常忽略智能体扩大后的攻击面和下游后果 [54,64,89,158]。过去面向智能体的研究通常只看某一种攻击向量 [185]、某一类智能体 [74,81] 或单个组件 [58]；设计导向的工作虽然提出高层原则，却没有系统整理攻击和防御 [15,186]。与本文最接近的综述 [33,181] 会讨论跨组件交互，但防御分类仍大多局限于基于模型的技术。

本文的区别在于：它用统一的风险分类覆盖全部智能体组件及其交互，并以纵深防御框架同时纳入模型防御和系统级防御，从而给出更可操作的智能体加固指导。

3. 智能体 AI 的设计版图

3.1 设计组件

本文把 AI 智能体定义为：由传统软件组件与 AI 模型共同构成的混合软件系统。典型智能体包括以下组件。

大语言模型。 LLM 是智能体的“大脑”，接收并分析用户任务，制定逐步计划，再依计划采取动作。整个过程可以由一个 LLM 完成，也可以由多个针对不同角色定制的 LLM 协作完成。

记忆。 记忆保存智能体的内部知识库和历史动作轨迹。这些信息通常会被向量化，以便高效检索，让智能体能够根据已有知识和过去经验作出后续决策。

工具。 工具是智能体与外部环境交互所调用的函数。按用途通常分为两类：检索工具用来从外部环境收集信息，例如搜索网页、读取文件；执行工具用来改变环境，例如写文件、发邮件、运行命令。工具可以通过 MCP [7] 等标准协议提供，既可能由智能体开发者编写，也可能来自第三方 [99,119]。

外部环境。 外部环境是智能体为完成任务而交互的上下文。环境类型很大程度上决定了攻击面和防御重点：

- 网页智能体会处理来自不同网站的任意不可信内容，因此尤其容易受到间接提示注入和数据外泄攻击；
- 编程智能体在 IDE 和本地文件系统中工作，主要风险是未授权代码执行和工作区篡改；
- 计算机使用智能体通过辅助功能 API 或屏幕界面操作多个应用，容易受到 UI 注入，并且往往拥有很广的系统权限；
- 智能体通过工具访问数据库等具体环境，例如读取或写入 SQL 数据库。

整体结构。 智能体接收用户查询后，会自动执行一串动作。工作流负责协调其中的 LLM 组件，而 LLM 通常承担两类角色：规划器把用户任务拆成步骤，执行器通过调用工具、生成内容或查询记忆来完成每一步。简单系统里，一个 LLM 可以同时承担两种角色；复杂系统则使用不同 LLM 分别规划和执行，以得到模块化控制和更细粒度的安全边界。

例如，用户要求更新文件时，规划器先确定步骤；执行器依次调用读文件工具、生成新内容，再调用写文件工具完成更新。这一结构可以自然扩展成多智能体系统，由多个专门化智能体共同完成复杂任务。

[图 1：AI 智能体的整体结构] 原图展示了用户、用户界面、LLM/工作流、记忆、工具和外部环境之间的关系。动作包括读取/检索、写入/更新/响应和执行；记忆包含会话轨迹、长期存储和内部 RAG；外部环境包括网页、计算机、特定领域环境、第三方服务和外部 RAG。

智能体系统与传统系统的区别。 第一，智能体把传统程序与 AI 推理结合起来，而传统软件主要依赖符号逻辑，因此前者更灵活、更能适应环境。第二，传统软件的工作流基

本由开发者预先编排，智能体却可以根据任务和输入上下文动态决定工作流与动作。第三，智能体常用向量记忆做语义检索，而传统系统一般通过预定义查询访问结构化存储。

3.2 七个设计维度及其安全含义

作者从上述结构中抽象出七个连续的设计维度。论文为了便于展示，把每个维度分成三个代表性等级，但真实智能体可以处在连续谱上的任意位置。

维度	灵活性最低	中等灵活性	灵活性最高
输入信任度	不使用外部数据	只使用预定义外部数据	使用任意外部数据
访问敏感度	不接触敏感数据	接触预定义敏感数据	接触任意敏感数据
工作流	简单聊天机器人	开发者定义的固定流程	LLM 动态定义流程
动作	只输出 LLM 回复	回复加检索	回复、检索和执行
记忆	无记忆	临时会话记忆	跨会话持久记忆
工具	无工具	固定且已知的工具	任意工具
用户界面	纯文本	网页图像预览	可交互网页元素等多模态界面

输入信任度。 衡量智能体依赖的外部数据源是否可信。从不使用外部数据，逐渐扩展到任意、可能不可信的数据源。数据源越丰富，知识和决策能力越强，但遭遇被攻陷或恶意数据源的风险也越高。

工作流。 指动作序列以及由谁决定这些序列，可类比传统软件中的代码。它从简单聊天、开发者固定编排，一直发展到 LLM 动态编排。灵活性使执行从僵硬的预定序列变成依上下文规划的流程，同时也给控制流劫持留下更多空间。

访问敏感度。 指智能体能访问多敏感的数据。权限从完全不接触敏感数据，扩展到已知敏感源，再到任意敏感数据。访问范围越广，能完成的复杂任务越多，但安全事件的潜在影响也越大。处理姓名、邮箱、财务记录、医疗数据等个人身份信息的智能体，尤其容易通过工具输出、日志或外泄攻击泄露信息。

动作。 指文本生成之外可以执行哪些操作。能力从只回复文本，扩展到检索、查询等只读操作，最后到编辑文件、执行命令、调用 API 等会改变环境的操作。动作越强，任务能力和潜在破坏力同时上升。

工具。 指可调用外部工具的范围，从不用工具，到调用固定、经筛选的工具，再到选择甚至使用任意工具。工具越开放，功能越强，攻击面也会显著扩大。

记忆。 指如何长期保存和检索信息。从无记忆，到会话级临时记忆，再到跨会话持久记忆。记忆有助于个性化和长期推理，但也会带来记忆投毒和隐私泄露。

用户界面。 指用户与智能体的交互形式，从纯文本，到图像预览或简单网页界面，再到网页、终端、IDE 等多模态交互。界面越丰富，工作流表达力越强，同时可利用的攻击向量也越多。

总体上，**灵活性与安全性存在明显权衡**。简单聊天机器人主要只有输入数据和模型参数两个攻击目标，攻击路径也基本是“用户输入 → 模型输出”。多智能体系统则可能同时暴露 LLM、共享记忆、工具和外部环境，组件之间也形成新的攻击路径，如 LLM 到工具、记忆到 LLM。复杂记忆和界面增加攻击机会；更开放的输入空间更容易被注入恶意数据或指令；更灵活的工作流则增加控制流被劫持、执行被重定向的概率。

4. 智能体 AI 的攻击版图

4.1 攻击向量

论文按照**攻击真正执行时**攻击者对智能体拥有的访问权限，划分三种威胁模型。攻击开发阶段需要的额外假设，则在攻击方法部分单独讨论。

外部攻击者

攻击者位于外部环境，不能直接与智能体交互，但可以修改智能体将来可能检索和处理的外部资源。这是权限最受限、却很现实的一类攻击者。

- **V1 间接提示注入** [53,60,73,83,160,167]: 把恶意指令写进公开网页、文档等外部环境。智能体读取内容时，也可能把这些指令取回并执行。
- **V2 恶意数据注入** [120,147]: 注入并非提示词的恶意数据。当这些数据参与敏感操作时，会触发安全故障，例如恶意软件包 [147] 或攻击者控制的金融参数 [120]。
- **V3 工具投毒与篡改** [108,138]: 把恶意指令放入外部工具的名称或描述中，或直接在工具实现中植入恶意载荷。

用户级攻击者

攻击者能够访问智能体输入，可以直接提交恶意内容 [88]，或把恶意内容混入表面正常的用户输入 [41,42]。

- **V4 直接提示注入** [41,42,88]：控制输入的一部分，把恶意指令附加到用户输入中。

内部攻击者

攻击者能够接触智能体内部的部分或全部组件。这是最强的假设，威胁最严重，但现实可行性相对较低。

- **V5 模型投毒** [159,175]：在 LLM 中植入后门，在推理阶段由触发条件激活并产生恶意行为。
- **V6 记忆投毒** [24,37,189,195]：直接操纵智能体记忆，写入恶意指令或虚假知识；也可能从记忆中窃取敏感用户数据。

[图 2：攻击向量 V1-V6 与安全风险 R1-R7 在智能体结构中的位置] 图中把外部环境、工具、用户界面、LLM 和记忆连接起来，展示不同攻击从哪里进入，以及可能触发哪些风险。

4.2 七类安全风险

R1 异构且不可信的接口。 与独立 LLM 相比，智能体向用户和环境暴露了更多异构接口：会被检索和处理的外部数据源、持续积累的持久记忆、可信程度不一的第三方工具等。攻击者可以把这些接口当作入口，攻击面远大于单一模型。

R2 遵循了错误指令。 智能体没有遵从正常用户或系统开发者的意图，反而执行了攻击者注入的恶意提示。根本原因在于：外部或攻击者控制的输入同样会被内部模型处理，因而能够影响模型行为。

R3 不受约束或不安全的数据流。 LLM 的随机性使其数据流缺乏硬约束。在传统系统中，编程语言、类型系统和访问控制会约束数据传播；在智能体中，来自不可信接口的任何输入，都可能自由流向用户回复、后续工具调用等任意输出。

这会进一步导致私密数据泄露（R5）、数据破坏（R6）和资源耗尽（R7）。例如，智能体读取不可信网页后，在回复中生成一个含攻击者控制数据的 URL；如果系统自动访问这个 URL，敏感数据便可能通过请求参数被带走 [96,128]。

R4 幻觉和模型错误。 模型会生成错误信息，而智能体会把幻觉内容付诸行动，使后果从“信息不准确”升级为真实世界中的错误访问和破坏。软件包幻觉攻击 [147] 就利用了这

一点：攻击者注册 LLM 经常凭空编造的软件包名，把模型缺陷转化成稳定的代码注入和供应链攻击路径。

R5 私密数据泄露。 智能体会在用户对话、持久记忆、工具凭据和环境资源等多个组件中处理敏感数据。攻击者可利用组件间交互窃取这些数据，例如诱导智能体把信息发往攻击者服务器；触发自动 URL 访问，通过查询参数外泄数据；或利用 SSRF、XSS、路径遍历、SQL 注入等领域漏洞访问受保护资源。近期事件已经造成真实的机密性破坏，暴露了聊天历史和个人信息 [101,110,132]。

R6 非预期/未授权动作与数据破坏。 这两类风险都会未经授权地改变内部或外部状态。非预期动作可能造成不可逆状态变化，例如擅自购物、执行任意代码；数据破坏则直接修改文件、数据库等资源。界面层可能向用户提供错误或误导性信息；记忆层可能写入有毒知识或恶意指令 [24]，持续污染后续行为；环境层则可能通过命令注入、SQL 注入或文件系统操纵执行未经授权的修改。

R7 资源耗尽与拒绝服务。 智能体会自主消耗算力、API 调用额度和外部系统资源 [70]。攻击者可以诱导昂贵 API 调用、制造无限执行循环或触发过量内存占用，使智能体及其外部系统不可用，或者在经济上无法持续。

此外，攻击者还可能以智能体系统本身为目标，窃取包含专有知识的系统提示、工具描述和配置 [137,176]，损害开发者和工具提供商。可观察的工具执行时序、API 调用序列或网络流量也可能形成侧信道。作者尚未发现专门研究智能体侧信道攻击的工作，因此把它留作未来方向。

攻击向量与风险并非一一对应。R2 可以由所有攻击向量触发；R5 可能来自 V1、V2 或 V6；R6 常由 V1 发起，也可能由 V3、V4、V5 触发；R7 则可能由任何外部或用户级向量引起。

4.3 系统级风险分析

[图 3：七个设计维度与七类风险的关系，以及风险之间的连接] 输入信任度、记忆和工具主要连接 R1；工作流主要连接 R2-R4；用户界面、访问敏感度和动作主要决定 R5-R7 的后果强度。

从设计维度到风险。 输入信任度、记忆和工具直接扩大攻击面（R1）：更多外部数据源带来间接提示注入，持久记忆成为投毒目标，第三方工具引入供应链风险。工作流主要推动模型风险（R2、R3、R4）：从聊天机器人走向 LLM 动态执行后，可变控制流更容易被劫持，错误指令和无约束数据流更严重；动态选择工具和资源时，幻觉会直接触发真实动作。

访问敏感度、动作和用户界面决定后果风险（R5、R6、R7）的严重程度。敏感数据越多，智能体一旦被攻陷，可泄露、破坏或耗尽的资源价值越高。动作从“只回复”扩大到“可执行”，会把信息暴露升级为环境破坏。复杂界面还会引入自动访问 URL、执行破坏性命令和界面操纵等新渠道。

风险的级联与放大。 R1-R7 会沿组件链条级联。攻击面扩大（R1）让攻击者控制的数据进入系统，再触发 R2-R4。比如，外部数据中的间接提示注入先让模型遵循错误指令，之后模型风险又放大为数据外泄、数据破坏或资源滥用。无约束数据流会增加泄密和恶意代码执行；幻觉则会让智能体操作虚构资源，进一步导致泄露和破坏。

EchoLeak [96,125] 是一个具体例子：企业邮件里的恶意文档利用异构不可信接口（R1），让数据不受约束地穿过智能体检索管线（R3），最终在用户完全不交互的情况下，把敏感数据送到攻击者服务器（R5）。

4.4 攻击方法

攻击方法负责构造攻击路径和攻击载荷。由于攻击链复杂，当前方法大多高度依赖人工 [95,96]。即时注入和记忆投毒的注入点几乎总由人手工选择；提示注入载荷也主要基于预设模式生成，例如角色扮演 [32]、用特殊字符制造分隔符混淆 [164,165]，或使用“忽略前文”一类指令重置上下文 [121,134]。

近期研究开始自动生成载荷和寻找注入点。提示注入方向出现了面向智能体的专用模糊测试 [160,180]，以及训练小型攻击模型生成恶意指令的方法 [83,167,194]。记忆投毒则同时追求“更容易被检索”和“内容看起来可信”。其中研究最系统的是语义注入：通过优化嵌入，让事实错误的内容在向量空间中贴近目标查询，从而在相似度检索中优先返回 [195]。更先进的方法使用基于梯度的优化，在维持语义连贯的同时，让对抗段落获得尽可能高的嵌入相似度 [87,184,189]。

总体而言，社区仍然需要更多端到端、自动化的智能体攻击与红队方法，供开发者在内部测试中使用。

5. 智能体 AI 的防御版图

相较攻击研究，智能体的完整防御版图仍缺少系统探索。本节先确定安全目标，再把防御分为运行时保护、安全内生设计、身份与访问管理、组件加固四类，最后总结设计原则。

[图 4：智能体防御版图] 原图把防御机制标在智能体内部与边界上：输入/输出护栏、信息流控制与污点跟踪、监控、人在回路验证、安全隔离与形式化验证、身

5.1 安全目标

前三项目标来自传统安全的 CIA 三要素，作者另为智能体引入“上下文安全”。

机密性。 保证信息只对获授权实体可见，包括 API 密钥和凭据等系统秘密、智能体内部记忆、用户私密数据，以及模型参数和系统提示等 LLM 相关数据。它主要缓解不安全数据流（R3）和私密数据泄露（R5）。

完整性。 保证智能体及外部环境中的数据流和控制流可信，未被未授权实体修改。保护对象包括记忆、LLM 输出、工具结果和环境数据，也要防止恶意指令诱导智能体执行非预期动作。不同智能体的具体完整性目标会随组件、数据流和控制流而变化。完整性覆盖错误指令（R2）、不安全数据流（R3）、幻觉与模型错误（R4），以及未授权动作和数据破坏（R6）。

可用性。 防止拒绝服务和资源滥用（R7），包括消耗 LLM token，以及滥用智能体宿主机和外部系统资源。

上下文安全。 近期研究 [139,152] 把它视为智能体特有的重要目标：执行过程中所使用的上下文必须与用户真正的任务对齐，不能被攻击者操纵。它要决定哪些上下文元素可以进入系统——例如系统提示、用户目标、工具描述、检索片段——以及如何确定优先级，从而避免指令覆盖、上下文漂移和不安全工具选择。它补充了主要保护数据正确性和秘密性的机密性、完整性目标。

上下文安全源自“上下文完整性” [12]，后者按社会规范和具体场景判断信息流是否恰当，已经用于普通程序 [142]、移动应用 [163] 和 IoT 系统 [65]。AirGapAgent [11] 把它用于智能体，只允许访问与当前任务上下文有关的信息。但有研究指出 [141]，许多借用上下文完整性的工作其实简化了信息流，没有充分考虑这一理论所要求的广泛情境和社会规范。智能体开放的输入和多样环境让“上下文与规范应如何定义”变得更困难。上下文安全把这一隐私概念扩展到安全领域，要求智能体行动持续符合用户意图，不被对抗输入扭曲。

5.2 运行时保护

运行时保护在智能体执行期间动态检测并拦截实时威胁。

5.2.1 输入护栏

输入护栏检查、清洗用户输入、工具返回和记忆数据等可能进入智能体的内容，是抵御外部攻击者和用户级攻击者的第一道防线。独立模型中的护栏主要检测越狱提示和有害内容 [25,61,90,136,140]；这些方法在智能体场景中仍然有效，尤其适合检查外部环境输入。智能体专用护栏还要处理动态检索和工具执行带来的风险，例如使用 URL 白名单 [51]，只允许从可信来源取回数据。

它有三个设计维度：

1. **检测机制。** 规则方法使用预定义模式 [126] 或端点白名单 [50,51]，严格但不灵活；模型方法训练小模型 [25,61,90,136] 或提示 LLM [138] 检测恶意内容。LLM 通常泛化和防护能力更好，但延迟与开销更高 [158]。
2. **检查目标。** 基于内容的护栏 [25,61,136,140] 检查输入语义是否恶意；基于来源的护栏 [51] 验证数据来源是否可信，特别适合网页搜索、数据库等动态外部检索。
3. **处置策略。** 可以检测后直接过滤 [25,61,90,136]，只清洗恶意片段 [140]，或用类型系统和验证规则把输入规范化为结构化格式 [193]。还有一些模型方法不显式判恶，而是扰动或平滑输入以消除对抗扰动 [129,190]。

开放问题在于，输入空间巨大而多样，很难建立统一安全标准。模型检测器会被自适应攻击绕过 [6,105]；规则系统需要大量人工维护、泛化性差。误报会损害可用性，漏报则让攻击者进入系统。更根本的是，“安全/不安全的数据或指令”本身经常没有清晰边界。

5.2.2 输出护栏

输出护栏检查智能体向外发送的结果，包括用户回复和会影响外部系统的工具调用。某些攻击在输入处看似无害，却会产生恶意动作 [96]，因此输出护栏可作为输入护栏的补充。

通用 LLM 输出护栏多用分类器或可编程规则检测有害文本 [93,100,126,136]。智能体专用方法还会验证工具使用和动作序列 [23,63,92,139,152,172]，确保它们符合具体上下文政策和用户意图。多智能体系统中，输出护栏也可以约束智能体间通信 [1]，或者限定允许的控制流图，阻止未经授权的智能体跳转 [62]。

输出护栏有两个主要维度：

- **检测目标：** 有害文本、危险代码、危险工具动作、是否符合用户意图、数据隐私、领域政策，以及运行时上下文安全。
- **检测机制：** 从简单规则匹配 [93,100]、模型分类器 [25,126,136]，到“结构化政策 + 模型处理非结构化数据”的混合方法 [23,29,92,139,152,172]。

其局限与输入护栏相似：安全标准模糊导致误报和漏报，规则与模型之间存在权衡。此外，输出护栏必须等待并处理 LLM 输出，因此通常更耗算力和时间 [158]。关键问题是

如何减少延迟和误报，在安全与效用间取得平衡。

5.2.3 信息流控制与污点跟踪

信息流控制（IFC）[13,16,34,102] 和污点跟踪 [107] 用于限制系统内的数据传播。基本思路是：从预定义安全格中给每份数据赋予标签 [34]，沿智能体执行过程传播标签，一旦信息流违反格上的约束便加以阻止。

已有方法同时覆盖完整性和机密性：完整性保护阻止不可信输入影响工具调用决策 [67,188,192]；机密性保护防止敏感数据流向不可信的汇点 [28,31,78,157]。

机制大致分三类：

- **多次执行：** 分别在包含和删除某输入的条件下多次推理，比较输入对输出的影响 [143,192]。
- **符号变量：** 用可追踪变量替换实际数据，避免所有内容都被过度标污，同时保留确定性的信息流保证 [28,31,67,78]。
- **模型判断：** 把智能体轨迹交给 LLM，让其检查信息流 [80,156,157,188]。

现有 IFC 和污点方案常有很高的运行开销，不适合低延迟智能体；保守标签还会不断扩散，最终让整个智能体都“有污点”而无法使用。未来需要轻量跟踪，以及有原则、可自动化的安全降级/解密规则，在不破坏安全性的前提下适度放宽标签。

5.2.4 监控

由于智能体行为动态且难预测，监控要同时观察输入、输出和中间状态，提供全局可见性。很多分布式攻击中，单个输入或动作看起来无害，合在一起才体现恶意目标 [162,182]；长流程和多智能体场景尤其需要跨工具、跨服务观察。

监控系统的检测目标包括长轨迹异常 [57,91,103] 和多智能体交互图异常 [191]。日志可以只记录动作和工具调用摘要，也可以保留中间推理与参数。粒度越细，检测能力越强，但存储、计算和隐私代价也越大 [19]。

监控同样受运行时防御的准确率和性能问题限制。智能体行为具有随机性、强依赖上下文，很难区分正常与恶意。静态规则看不到新攻击，自适应模型则开销高且仍可能被绕过。长时间执行还会积累大量日志，相应的存储和隐私控制尚缺少研究。

5.2.5 人在回路验证

传统系统会在安装应用 [38] 或访问敏感数据 [9] 时征得用户同意。智能体中的人在回路验证让用户检查智能体行为和工具使用，并按自己的需求控制安全决策。专门研究虽然不多，但 GitHub Copilot、Gemini Code Assist、Cursor、Codex 等编程智能体已经会

在写文件或执行命令前请求批准。许多防御系统 [28,31,67,170] 也会在动作违反政策时交给人类决策。

它有三个设计维度：

1. **验证范围：** 哪些动作需要批准。常见范围包括终端命令、工作区外文件访问和删除等破坏性操作。
2. **警告内容：** 应说明智能体准备做什么、风险是什么、可能造成什么后果，帮助用户真正理解后再决定。
3. **重复策略：** 为减少批准次数和决策疲劳，可以记住用户选择，类似移动权限中的“仅本次允许”“永不允许”“始终允许”。研究 [171] 还用机器学习建模用户偏好，预测决定以减少提示次数。

主要风险是频繁弹窗让用户疲劳，反而削弱安全性。当前警告往往假定用户具备较高安全素养，结果可能是盲目批准危险操作，或过度拒绝正常操作。实际系统需要更有原则地决定何时交给人类判断，同时给出信息充分但轻量的解释。

5.3 安全内生设计（Secure by Design）

这类方法在架构层直接建立安全属性，让安全成为系统本身的一部分。由于它依赖具体架构，而智能体与传统系统存在根本差异，因此尤其具有智能体特色。

5.3.1 权限分离

权限分离遵循最小权限原则，把不同权限赋给不同组件并相互隔离，以缩小整体风险。

分离政策分为垂直和水平两种：

- **垂直分离**建立权限层级，高权限组件（如可信规划器）比低权限组件（如不可信数据处理单元）有更大权力。类似操作系统的内核态/用户态分离，规划器—处理器架构 [5,28,31,67,79,166,168] 把高权限的工具调用规划，与低权限的工具结果处理隔开；记忆最小化 [11] 则把高权限的数据最小化器与不可信数据处理单元分开。
- **水平分离**把系统拆成权限相同但访问范围隔离的并行组件。例如，为每个应用或功能设置独立智能体 [78,170]，只允许访问各自工具和资源，防止攻击跨领域蔓延。

分离对象也可以不同：把同一个 LLM 的工具规划和结果处理分开，避免工具返回直接影响计划 [28,31,67,168]；建立不同记忆空间，防止低权限组件看到敏感数据 [11]；使用沙箱、容器 [84-86] 和访问令牌 [52,145] 隔离外部环境。

当前研究大多只在简化环境中处理通用间接提示注入 [32]，没有覆盖网页、文件系统、数据库等真实环境的复杂风险。组件被隔离后，系统功能和效用也会下降。如何把智能

体恰当地拆成一组最小权限组件，并让它们高效通信而不损害安全保证，仍是开放问题。

5.3.2 用形式化验证获得可证明安全

传统形式化方法能为程序正确性和安全性给出理论证明 [56,69]。对智能体来说，这仍是一个刚起步但重要的方向：如何把符号化保证与非符号模型行为结合起来。

VeriSafe Agent [76] 把用户意图转成描述 UI 状态迁移的 DSL，在动作真正执行前验证 GUI 操作是否符合任务。Formal-LLM [82] 用下推自动机编码开发者定义的计划约束，例如工具必须按特定顺序调用，从而限制计划生成并确保计划有效、可执行。现有验证属性包括预设安全约束 [23]、与用户任务的一致性 [76]，以及行为是否符合函数需求和预期输出 [82]。

困难在于，传统形式化方法面对的是结构化语言编写的确定性程序，LLM 智能体却基于概率模型。要描述随机行为又保留有意义的安全保证并不容易。与网页、文件系统或移动应用交互时，应满足的安全属性本身也定义不足。建立覆盖智能体完整安全需求的验证框架仍然是重大开放问题。自动化对这一方向尤其重要，前沿 AI 也许可以帮助自动生成形式化规格 [174]。

5.4 身份与访问管理

身份与访问管理（IAM）包含身份管理、访问控制和凭据管理，用来保证实体身份真实、资源访问经过授权。传统 RBAC [131] 和基于 OAuth 的委托 [39,59] 通常使用静态用户身份和预定义权限边界；智能体会代表用户访问现实服务，因此还需要智能体专用身份、委托机制，以及能随运行时上下文变化的动态访问政策。

5.4.1 身份管理

身份管理通过认证和授权确保每个行为主体以正确身份行动，是访问控制的前提。智能体身份还应支持委托、审计和监管问责，并与现有标准 [39,59,154] 对齐。

其设计维度包括：

- **架构：** 中心化方法 [146,148] 依赖中心注册表或基于 OpenID Connect [39] 的身份提供商 [27,109]；去中心化方法使用分布式验证协议。ANP [20] 采用 W3C DID 标准 [154]，Microsoft Verified ID [94] 支持点对点身份验证。后者自主性和韧性更强，但实现和协调更复杂。
- **身份粒度：** 可以是用户级、智能体级或任务级。用户级把动作直接归因于人；智能体级给每个智能体独立身份 [19]；任务级则为具体任务或会话创建短期身份，以支

持动态最小权限。

- **委托模型：**直接委托把明确的用户权限交给智能体；代理委托通过中介服务或令牌让智能体代表用户；临时委托则自动过期，缩短风险窗口。

目前仍没有统一答案：动作责任应归于人类操作者、智能体实例，还是临时任务身份？不同领域答案也可能不同。平台各自实现不兼容的凭据签发、委托和撤销流程，难以审计和联合。未来需要可互操作的分类与生命周期工具，在顺畅协作的同时保留问责能力。

5.4.2 访问控制

智能体用户的私有资源既可能在记忆中，如使用历史和个性化知识库，也可能在外部环境中，如文件系统和云盘。访问控制通过限制智能体能使用的工具和数据源来保护这些资源。

机制可以是 RBAC [177,187]、ABAC [4] 或基于能力的安全系统。对向量数据库，可依据模型激活模式授权 [177]、过滤输出 [4]，或直接分区数据库，只暴露允许访问的条目 [187]。访问外部网页服务时，可结合 API 密钥、OAuth 2.0 [59] 和安全委托协议 [146]。

多智能体系统还必须防止“混淆代理”攻击 [55]：某个智能体借助另一个智能体的能力，非法获得更高权限。研究 [148] 用密码协议执行用户定义的多智能体通信政策，控制智能体间访问。

政策既可以是静态规则，也可以依据任务、上下文和环境状态动态调整。受保护资源包括内部记忆、外部数据库、工具 API 和文件系统。

目前研究主要集中于带向量数据库的 RAG 应用 [4,177,187]，尚未覆盖智能体的多种工具、数据源和跨智能体交互。实际部署缺少自适应政策框架，常靠零散、不统一的配置，造成智能体间政策错配，或与非智能体服务协作时留下缺口 [127]。配置过于复杂还会导致误配置和权限过大，即使技术用户也很难避免。

5.4.3 凭据管理

智能体为了访问第三方服务和用户资源，需要处理 API 密钥、访问令牌、邮件一次性密码、会话令牌等多种凭据，因此面临显著隐私与安全风险 [106,149]。

凭据管理覆盖三个方面：

1. **机密存储。**加密静态凭据；只在会话期间临时保留；使用独立令牌保险库，让智能体代码永远接触不到真实凭据 [10]；或在硬件可信执行环境中用机密计算保护凭据

[72,135]。论文也以 OpenAI 临时聊天 [116] 作为“不保存聊天历史、不用于训练”的临时存储例子。

2. **生命周期。** 凭据可以在整个会话中保持不变，也可以使用会自动过期的动态短期令牌，缩短暴露时间。
3. **凭据发放。** 包括 SSO [27]，以及基于 OAuth 2.0 的认证委托 [146]，用于从用户安全地向智能体转移权限。

现实中的智能体栈仍常用临时拼凑的秘密处理方式。许多框架甚至指导开发者把 API 密钥以未加密环境变量保存，增加泄露风险。多智能体流程中的秘密共享更难兼顾最小权限和可追踪性，因此需要协调一致的凭据编排。

5.5 组件加固

组件加固分别增强模型和工具，因为整个系统的安全性取决于最弱的组件。

模型加固。 SecAlign [22] 和 StruQ [21] 通过微调，让模型遇到冲突指令时仍遵循最初指令。理解指令层级的训练 [155,169] 则确保系统提示的优先级高于用户输入和外部数据。

工具加固。 ETDI [36] 为工具控制元数据加入密码签名和版本管理，保证工具整个生命周期的完整性，防止工具投毒。MCP Context Protector [151] 作为 MCP 代理，强制人工审查并检查工具描述与响应。MCP Safety Audit [124] 系统审查工具，发现恶意逻辑或误导性描述中的可利用行为。MCIP [66] 为 MCP 增加可观测性，并用专门微调的 LLM 检测威胁。Anthropic Connectors Directory [8] 则维护经过描述、功能和安全政策审查的可信工具目录。

当前加固方法多面向简化威胁，不能反映真实智能体的复杂性。模型的指令层级训练主要处理系统提示、用户提示和工具结果之间的简单冲突，却难以应对“影子攻击”——恶意工具结果覆盖其他工具结果。不同组件应该承担什么角色、适用何种完整威胁模型，仍不清楚。

5.6 防御设计原则

有效的智能体安全不能依赖单一方法，而要让多种机制相互补充。作者强调三个来自传统安全、但在智能体中更加重要的原则。

纵深防御。 输入护栏过滤恶意输入，输出护栏作为最后一道动作防线；信息流控制和监控贯穿执行过程；访问控制负责认证与授权；权限分离从架构上建立安全边界；组件加固提高单个元素的安全性；人在回路为关键决策提供监督。

不过，叠加防御也可能产生“涌现式错位” [14]：一种机制无意间削弱另一种。例如，前置清洗器可能删掉下游护栏所依赖的安全指令。因此纵深防御必须在整个智能体栈上协调设计，而不是简单堆叠。

最小权限。 智能体只应获得完成具体任务所必需的权限和访问权。权限分离负责限制每个组件和工具的能力，身份管理则定义适当的访问范围。

完全仲裁。 每一次对敏感资源的访问都必须被检查和授权。监控、访问控制和身份管理共同验证智能体与受保护资源的每次交互。

[表 2：防御机制与所覆盖风险] 原表把方法映射为：输入护栏主要处理 R1；输出护栏覆盖 R2、R4、R5、R6；信息流控制覆盖 R3、R5、R6；监控与人在回路覆盖 R5-R7；权限分离和形式化验证主要覆盖 R2、R3；身份与访问控制覆盖 R5-R7；凭据管理覆盖 R5；模型加固覆盖 R2、R4；工具加固覆盖 R1。表中还列出了各类别的代表论文。

6. 真实智能体如何做安全防护

作者分析了六个开源智能体，观察真实系统如何组合第 5 节中的防御方法。这里只考察系统级防御，不包含组件加固。论文把“需要明显人工配置、人工筛选，或只覆盖部分威胁”标为部分支持；把“自动完成并覆盖完整范围，即使准确率不完美”标为完整支持。由于这些项目都在快速迭代，分析只代表论文写作时的版本状态。

[表 3：真实智能体的防御覆盖情况] 对比对象为 Codex v0.53.0、Gemini CLI v0.13.0、OpenHands v0.59.0、Browser Use v0.9.0、Nanobrowser v0.1.12 和 Skyvern v0.2.20。比较维度包括输入/输出护栏、访问控制、信息流控制、监控、人在回路、权限分离、形式化验证、身份管理和凭据管理。原表用实心/空心标记表示完整或部分支持，建议需要精确版本对比时直接查看 PDF 第 20 页。

6.1 编程智能体

编程智能体通常在用户信任的目录中工作，但威胁并未消失：LLM 可能幻觉并生成错误代码；模型可能被植入后门（V5）；用户输入、代码仓库和文档等表面可信的内容也可能藏有恶意指令（V1、V4），而用户并不知情。

因此，编程智能体通常不优先清洗输入，而是重点限制动作。例如，它们通过输出护栏和访问控制约束 AI 生成的文件系统操作与 shell 命令；敏感动作是否安全高度依赖上下文，所以还大量依靠人在回路验证。三个编程智能体的监控能力都只有部分支持：它们

通过 OpenTelemetry [117]、PostHog [123] 等记录工具调用和文件修改，供事后人工审计，但没有自动检测可疑模式或异常行为。

Codex

论文所考察的 Codex [115] 结合访问控制与人在回路验证，保护 AI 提议的文件补丁和 shell 命令，同时部分支持输出护栏和监控。

访问控制通过路径限制和提权控制实现：如果补丁目标不在可写路径中，例如不在当前工作目录，就需要用户批准。shell 命令默认在无网络、只允许访问工作目录的沙箱中执行 [35]。确有需要时，模型可以申请在沙箱外以提升后的权限运行，但必须得到用户批准。这样，自动动作默认受到隔离，正当任务仍可在受控条件下提权。

Gemini CLI

Gemini CLI [49] 主要依靠命令执行前的人在回路验证，并部分支持输出护栏和监控。它限制文件读取只能发生在工作区内；shell 命令使用允许列表和拒绝列表，未列出的命令都要询问用户。系统可以缓存决定，并把复合命令拆成独立部分分别记录，减少重复询问。

不过，这仍然是规则型护栏，覆盖并不完整。底层环境最终依赖用户本身的操作系统权限，没有额外 OS 级访问控制。Gemini CLI 建议用户在 Docker 等容器中运行以隔离整个智能体 [45]，但搭建和维护沙箱也有明显成本。

OpenHands

OpenHands [3] 采用多层方案：输出护栏、人在回路和权限分离，再加部分监控。它要求 LLM 在每次工具决策时一并输出 **security_risk** 风险分数，以结合上下文发现高风险工具调用。高风险调用必须经用户同意才能继续。

与 Gemini CLI 类似，它不对每条命令单独做沙箱隔离，而是建议把整个智能体放进容器。权限分离则依靠多智能体架构：不同智能体拥有不同工具和能力，例如编程智能体可访问文件系统，浏览智能体可访问网络。这样，即使一个智能体被攻陷，影响也被限制在它的权限边界内，同时可以进行跨智能体安全委托。

未来方向

编程智能体还缺少多项关键防御：应使用输入护栏检查工作区、用户提示和网页检索内容；应用信息流控制追踪不可信数据如何影响工具调用；建立身份和凭据管理，对 API 密钥、访问令牌进行正确认证和安全存储。

现有防御也需要增强。访问控制不能只有“全部允许/全部拒绝”，而应在多种开发工具间提供顺畅的细粒度最小权限；批准提示应显示预期副作用、历史批准对比等更丰富上下文，减少决策疲劳；监控应在日志之上加入自动检测，发现领域特定风险和提权尝试，以使用户在损害发生前介入。

6.2 网页智能体

网页智能体可执行从页面总结，到打开 URL、点击按钮、填写表单等任务。它天然要处理来自不同来源的任意内容，即使任务涉及个人身份信息、支付数据，或访问云文件、发送邮件、购买商品等已登录流程，输入仍然可能不可信。

网页智能体尤其容易受到间接提示注入（V1）：网页里的恶意指令可劫持其行为。与主要在可信工作区运行的编程智能体不同，它持续暴露在巨大而多样的不可信输入面上，又经常代表用户访问私密数据和凭据，因此是高价值攻击目标。其他威胁还包括被攻陷的模型（V5）、模型幻觉和来自用户提示的直接攻击（V4）。

常见防御有四类：输入护栏用域名允许/拒绝列表或页面元素过滤来拦截恶意内容；输出护栏阻止导航到本地网络主机、裸 IP、`chrome://settings` 等敏感 URL；凭据管理在敏感数据进入 LLM 服务商或不可信域名前进行替换、脱敏；监控记录浏览遥测数据供事后检查。但所调查系统都没有把日志接到自动检测器上，大部分实现仍只是人工维护、覆盖有限的部分防护。

Browser Use

Browser Use [153] 支持输入/输出护栏、凭据管理和监控。输入侧使用广告拦截规则删除广告和其他不需要的页面元素，因此只能有限抵御恶意提示。凭据侧根据用户定义的“秘密—允许披露域名”映射，在内容到达第三方 LLM 或不可信域名前把秘密替换成占位符。

Nanobrowser

Nanobrowser [104] 支持输入护栏和权限分离，部分支持输出护栏、凭据管理和监控。它用分隔符和防护提示把用户指令与网页内容明确区分，减轻间接提示注入；还用正则表达式识别并脱敏社会保障号、信用卡信息、邮箱等敏感数据。

在权限分离方面，Nanobrowser 把职责拆给拥有不同权限的规划器和导航器，因此导航器即使被攻陷，也不能直接修改智能体总体计划。

Skyvern

Skyvern [144] 支持输入/输出护栏、凭据管理和监控，尤其重视凭据保护。它在运行时用正则识别一次性密码（OTP），先替换成占位符，不让 LLM 服务商和不可信网页看到真实值，只有填入认证表单时才恢复。此方法提高了 OTP 外泄难度，但正则覆盖不完整，也没有扩展到其他凭据类型。

未来方向

网页智能体同样需要纵深防御。当前产品仍偏早期，功能和防御都比较简单、脆弱。完全缺失的能力包括：用信息流控制追踪不可信网页内容如何影响决策；用身份管理支持智能体代表用户跨服务行动；对购买、分享数据等高影响动作引入人在回路。

输入护栏也不应只靠人工域名名单，而应使用自动域名信誉与上下文过滤，处理域名过期后被攻击者接管等风险 [130]。权限分离、污点跟踪和模型级输入护栏可以组合起来，在恶意指令抵达规划器前使其失效。监控应加入实时检测，在发现异常时暂停并升级到人工确认。凭据管理则需要更自适应、保护隐私的识别方法，覆盖简单正则之外的更多凭据。

7. AutoGPT 详细案例

AutoGPT [173] 是广泛使用的开源自主智能体，GitHub 星标超过 18 万。它提供大量工具，让 LLM 可以操作互联网、本地文件、命令行等异构环境。作者分析了从 v0.4.3 开始的多个版本、公开漏洞报告和相应防御。

7.1 工具与执行环境

检索工具： `google_search` 搜索网页，`browse_website` 获取页面内容，`read_file` 和 `search_files` 浏览本地文件，`load_from_memory` 读取历史上下文。

执行工具： `execute_shell` 执行任意 Bash 命令，`file_manipulation` 修改工作区，`execute_python_code` 动态执行 Python。

外部环境： 包括可通过搜索访问的互联网、可读写的本地文件系统，以及 Python 解释器、shell、OS 接口等领域环境。

7.2 五个真实漏洞

A. Docker Compose 文件覆盖：CVE-2023-37273

项目的 `docker-compose.yml` 没有写保护，因此恶意 LLM 输出可以覆盖容器配置。攻击者在智能体会读取的网页等外部内容中嵌入恶意指令，劫持 LLM 调用 `execute_python_code` 覆盖配置文件（R2）。AutoGPT 重启时会运行恶意容器，进而逃逸容器并攻陷宿主机（R6）。

B. 路径遍历：CVE-2023-37274

未经清洗的 `basename` 参数允许使用 `../../../../main.py` 一类路径，在沙箱外写文件。攻击者把指令注入外部内容，诱导 LLM 用遍历路径调用 `execute_python_code`，覆盖 AutoGPT 关键源码（R2、R6）。重启后，修改过的文件被执行，形成持久任意代码执行；被覆盖文件还可能暴露源码或配置中的敏感信息（R5）。

C. ANSI 转义序列欺骗：CVE-2023-37275

ANSI 转义序列是终端用于移动光标、清屏、改变颜色等操作的控制码。AutoGPT 用 `browse_website` 获取网页后，会在不清洗的情况下把内容直接送到控制台，其中也包括 ANSI 码。攻击者可制作包含 JSON 编码转义序列的网页。

这些序列不是劫持 LLM 的指令，而是未经清洗的数据，沿智能体管线流到终端并被解释（R3）。伪造的控制台输出可以隐藏已执行命令，或诱骗人类操作者批准恶意动作，悄悄劫持智能体行为（R6）。

D. 跨站请求伪造：CVE-2024-1879

缺少 CSRF 保护且 CORS 配置过于宽松，使恶意网页能够发起已经认证的 API 请求。用户访问恶意网页后，页面可以在后台用跨域请求触发智能体动作，造成未授权命令执行和数据外泄（R5、R6）。

E. 操作系统命令注入：CVE-2024-1881

AutoGPT 使用允许列表检查 shell 命令，但只验证第一个 token。它能拦截单条危险命令，却看不到操作符串联的载荷或一行中的多条命令。攻击者可以诱导 LLM 生成 `ls && rm -rf /、cat file; curl attacker.com` 一类组合命令（R2）。即使没有攻击者，LLM 也可能为了完成复杂任务自行生成串联命令，因幻觉或模型错误绕过首 token 检查（R4）。执行器会原样运行整条命令，导致任意代码执行、数据外泄和文件系统破坏（R5、R6）。

7.3 AutoGPT 的补丁与防御缺口

AutoGPT 在多个版本中修补了上述漏洞。一个共同问题是：补丁都在控制下游后果，例如加强访问控制或清洗输出，却没有处理上游根因，间接提示注入（R2）和不安全数据流（R3）仍然存在。

Docker Compose 覆盖。 v0.4.3 之后使用只读挂载，并限制配置文件权限，因此能阻止 `docker-compose.yml` 被覆盖，缓解 R6。但触发覆盖尝试的 R2 没有解决。输入侧应扫描网页提示注入，信息流控制应阻止受污染输出进入 `execute_python_code`。

路径遍历。 v0.4.3 之后在 `agent.workspace.get_path()` 中做路径规范化，并过滤 `../` 等模式。工作区根目录配置正确时，这能阻止基础遍历和 R6，但 R2、R5 仍然开放。输入护栏应从源头阻断注入，信息流控制则应追踪网页内容到文件系统操作的污点传播。

ANSI 转义欺骗。 v0.4.3 之后使用规则清洗模型输出中的转义序列，部分缓解 R3。但规则无法覆盖所有变体，可能被新编码或少见控制码绕过。更稳妥的做法是把所有外部网页数据都视为不可信，并在每一个输出边界——包括终端，而不仅是 LLM 上下文——强制清洗。

CSRF。 v0.5.1 之后加入 CSRF token，并执行只信任 localhost 端口的严格 CORS 政策。这能阻止大多数跨域攻击，但恶意浏览器扩展或能访问本机端点的本地应用仍可能攻击。身份管理应采用基于 OAuth 的授权，而不是只验证 token；监控则应追踪跨域访问模式。

OS 命令注入。 当前版本仍使用只检查首 token 的命令允许列表，无法阻止操作符串联或多命令载荷。输出护栏应真正解析 shell 语法；人在回路界面应展示命令风险；权限分离应按风险等级用最小权限隔离 shell；监控应记录命令来源；形式化验证可把执行限定在预先批准的命令模板内。

漏洞	被利用风险	已部署防御	已缓解	仍开放	缺失的关键防御
Docker Compose 覆盖	R2、R6	只读挂载/访问控制	R6	R2	输入护栏、信息流控制
路径遍历	R2、R5、R6	路径规范化/输出护栏	R6	R2、R5	输入护栏、信息流控制
ANSI 转义欺骗	R3、R6	正则过滤/输出护栏	部分 R3	R3 仍不完整	信息流控制

漏洞	被利用风险	已部署防御	已缓解	仍开放	缺失的关键防御
CSRF	R5、R6	CSRF token、CORS	部分 R5、R6	localhost 绕过	身份管理、监控
命令注入	R2、R4、R5、R6	首 token 允许列表	部分 R6	R2、R4、R5	更强输出护栏、人在回路、权限分离、监控、形式化验证

8. 结论

本文系统描绘了 AI 智能体的攻击与防御版图，并分析了智能体风险、安全目标、防御设计维度、真实案例和开放挑战。

综述表明，智能体安全研究在梳理问题空间方面已经取得明显进展，但真正通用、实用的防御仍然十分欠缺。接下来的关键方向包括：建立连接实验研究与生产环境的真实评估框架；设计能够安全组合、不会产生涌现式错位的防御机制；标准化智能体身份和访问控制；构建能在安全性与可用性之间动态平衡的自适应防御。

作者希望这套知识体系既能指导安全智能体的构建，也能帮助研究者找到真正有价值的后续问题。

参考文献说明

正文中的引用编号与原论文完全对应。原文共列出 195 项参考资料，其中包含论文、技术报告、产品文档、标准、项目主页与 CVE。为避免把作者名、论文题目、会议名和 URL 再翻译一遍反而降低检索效率，本阅读稿不重复抄录书目；查找某一编号时，请直接参阅原 PDF 的 References 部分。